

DOCUMENTACIÓN TÉCNICA PROFESIONAL

Ataque DoS | Protocolo CDP

Randy Nin | Matrícula: 2025-0660

Índice

- Contexto técnico
- Objetivos
- Alcance y uso ético
- Entorno del laboratorio
- Herramienta: cdp-flooding.py
 - 5.1 Requisitos del entorno
 - 5.2 Parámetros de ejecución
 - 5.3 Análisis técnico del script
- Desarrollo del ataque
 - 6.1 Estado inicial del sistema
 - 6.2 Ejecución
 - 6.3 Evidencia del impacto
- Mitigación aplicada
- Verificación post-mitigación
- Conclusión
- Referencias

I. Contexto técnico

CDP (Cisco Discovery Protocol) es un protocolo propietario de Capa 2 que permite a los dispositivos Cisco descubrir automáticamente sus vecinos directamente conectados, intercambiando datos como nombre del equipo, versión del sistema operativo, capacidades e identificador de interfaz. Opera sobre la dirección multicast `01:00:0c:cc:cc:cc` y viene habilitado por defecto en prácticamente todo dispositivo IOS.

La debilidad estructural de CDP es que no implementa ningún mecanismo de autenticación ni verificación de origen. El dispositivo acepta como legítimo cualquier frame CDP que llegue por sus interfaces activas, independientemente de quién lo emita. Esto permite a un atacante con acceso de Capa 2 inyectar paquetes CDP falsificados de forma masiva, forzando al router a procesarlos todos y saturar su tabla de vecinos, con el consecuente incremento descontrolado en el consumo de CPU.

2. Objetivos

2.1 Objetivo del laboratorio

Demostrar en un entorno controlado de GNS3 que un equipo con acceso directo de Capa 2 a la misma red que un router Cisco puede provocar una condición de denegación de servicio al saturar la tabla CDP del router con entradas completamente falsificadas, elevando su carga de CPU hasta el 100% en el proceso de interrupciones.

2.2 Objetivo del script

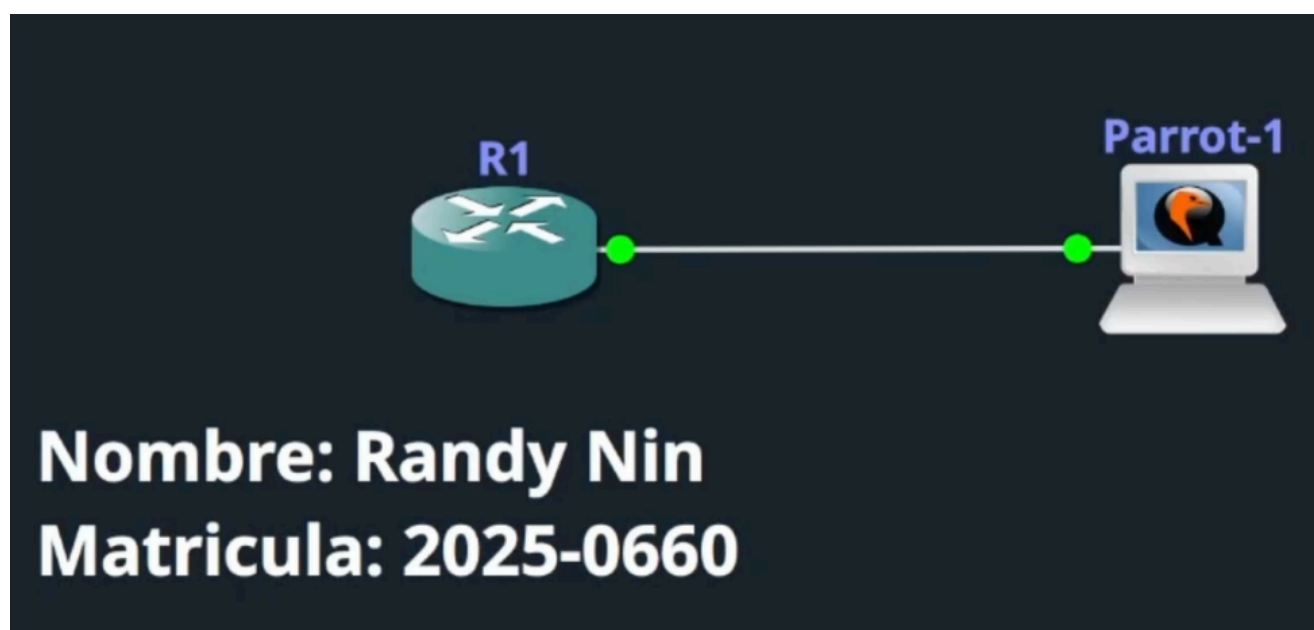
Automatizar la generación y el envío continuo de frames CDP falsificados hacia la dirección multicast CDP, utilizando identidades completamente aleatorias en cada iteración (Device ID, dirección MAC de origen y Port ID), con el objetivo de inundar la tabla de vecinos del dispositivo víctima y provocar un consumo de recursos que cause degradación severa o pérdida total del servicio.

3. Alcance y uso ético

Esta práctica fue desarrollada con fines exclusivamente académicos dentro de un entorno aislado en GNS3. Las técnicas y herramientas documentadas en este informe deben utilizarse únicamente en redes propias o entornos de prueba autorizados como GNS3, EVE-NG o PNETLab. Su aplicación en redes de producción, corporativas o de terceros sin autorización expresa constituye una violación legal y está completamente fuera del alcance de este laboratorio.

4. Entorno del laboratorio

La topología de este laboratorio es punto a punto entre el router víctima y la máquina atacante. CDP opera exclusivamente a nivel de Capa 2, por lo que no se requiere configuración de direccionamiento IP. El router R1 no tiene ninguna configuración previa aplicada al inicio del laboratorio.



Dispositivo	Rol	Interfaz	Sistema / Plataforma
R1	Víctima	Fa 1/15	Cisco 3725 / IOS 124-15.T14
Parrot-1	Atacante	ens4	ParrotSec OS

Nota: CDP está activo por defecto en Cisco IOS. No se requiere ninguna configuración previa en R1 para que el ataque funcione.

5. Herramienta: cdp-flooding.py

5.1 Requisitos del entorno

Sistema operativo: ParrotSec OS, Kali Linux o cualquier distribución Linux que soporte envío de paquetes raw a nivel de Capa 2.

Privilegios: El script requiere ejecución con permisos de superusuario (`sudo`). Si se intenta correr sin privilegios, el script lo detecta mediante `os.geteuid()` y termina con un error controlado.

Python: Versión 3.x.

Librerías externas requeridas:

Librería	Uso dentro del script	Instalación
<code>scapy</code>	Construcción y envío de frames Ethernet, LLC, SNAP y Raw	<code>pip install scapy</code>
<code>pwntools</code>	Indicadores de progreso en tiempo real con <code>log.progress</code>	<code>pip install pwntools</code>

Los módulos `signal`, `sys`, `random`, `string`, `os`, `struct` y `argparse` son parte de la librería estándar de Python 3 y no requieren instalación adicional.

Instalación de dependencias:

```
pip install scapy pwntools
```

Conectividad requerida: El sistema atacante debe tener conectividad directa de Capa 2 con el dispositivo Cisco víctima. CDP debe estar habilitado en la interfaz del router, que es el estado predeterminado en IOS.

5.2 Parámetros de ejecución

El script recibe un único argumento opcional desde la línea de comandos:

Parámetro	Flag	Valor por defecto	Descripción
Interfaz de red	<code>-i / --interface</code>	<code>ens4</code>	Interfaz de la máquina atacante desde la que se inyectan los frames CDP falsificados

Sintaxis:

```
sudo python3 cdp-flooding.py -i <interfaz>
```

Ejemplo utilizado en el laboratorio:

```
sudo python3 cdp-flooding.py -i ens4
```

La ejecución continúa de forma indefinida hasta que el usuario presiona `Ctrl+C`, lo que activa el handler de señal `SIGINT` y termina el proceso de manera limpia mostrando el mensaje de parada.

5.3 Análisis técnico del script

El script construye cada frame CDP desde cero utilizando Scapy y bytes crudos mediante `struct.pack`, sin depender de ninguna implementación CDP predefinida. Esto otorga control total sobre cada campo del protocolo, y permite calcular el checksum correctamente para que el router acepte los paquetes como legítimos.

Datos generados aleatoriamente en cada iteración del loop:

Campo CDP	Lógica de generación
Device ID	Prefijo fijo <code>RANDYN</code> + 8 caracteres alfanuméricos en mayúsculas aleatorios
MAC de origen	Formato <code>02:00:00:XX:XX:XX</code> con los tres últimos octetos aleatorios
Port ID	Formato <code>Fas X/X</code> con índices entre 0 y 3
Platform	Fijo: <code>cisco WS-C2960-24TT-L</code>
Software	Fijo: <code>Cisco IOS 12.2(55)SE</code> (imita un Catalyst 2960)
Capabilities	Fijo: <code>0x00000009</code> (bits Router y Switch activos simultáneamente)

Estructura del frame enviado en cada iteración:

```
Ethernet (dst=01:00:0c:cc:cc:cc, src=MAC_aleatoria)
├─ LLC (DSAP=0xAA, SSAP=0xAA, ctrl=0x03)
│   └─ SNAP (OUI=0x000000C, code=0x2000)
│       └─ CDP Raw
│           └─ Header v2 (version=2, TTL=180s, checksum calculado
con complemento a uno)
│               └─ TLVs: Device ID / Port ID / Software / Platform /
Capabilities
```

El checksum del header CDP se calcula manualmente sobre el header completo antes de cada envío, lo que garantiza que el router lo valide como un paquete CDP legítimo y lo incorpore a su tabla de vecinos. Dos indicadores de `pwntools` muestran en tiempo real el total de paquetes enviados y los datos del último frame inyectado.

6. Desarrollo del ataque

6.1 Estado inicial del sistema

Antes de iniciar el ataque, la tabla de vecinos CDP del router no contiene ninguna entrada con el prefijo `RANDYN` y el proceso `CDP Protocol` no aparece entre los de mayor consumo de CPU. El sistema opera en condiciones normales.

```
R1#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater

Device ID         Local Intrfce   Holdtme    Capability Platform  Port ID
R1#
```

```
R1#show processes cpu sorted
CPU utilization for five seconds: 0%/0%; one minute: 0%; five minutes: 0%
PID Runtime(ms)   Invoked    uSecs   5Sec   1Min   5Min TTY Process
  1         0         67         0 0.00% 0.00% 0.00% 0 Chunk Manager
  2         0         60         0 0.00% 0.00% 0.00% 0 Load Meter
  3         0          1         0 0.00% 0.00% 0.00% 0 chkpt message ha
  4         0          1         0 0.00% 0.00% 0.00% 0 EDDRI_MAIN
  5        16         29       551 0.00% 0.00% 0.00% 0 Check heaps
  6         0          1         0 0.00% 0.00% 0.00% 0 Pool Manager
  7         0          2         0 0.00% 0.00% 0.00% 0 Timers
  8         0          1         0 0.00% 0.00% 0.00% 0 OIR Handler
  9         0          1         0 0.00% 0.00% 0.00% 0 Crash writer
 10         0         11         0 0.00% 0.00% 0.00% 0 Environmental mo
 11         0          5         0 0.00% 0.00% 0.00% 0 IPC Dynamic Cach
 12         4          1      4000 0.00% 0.00% 0.00% 0 IPC Zone Manager
 13         0        293         0 0.00% 0.00% 0.00% 0 IPC Periodic Tim
 14         0        293         0 0.00% 0.00% 0.00% 0 IPC Deferred Por
 15         0          1         0 0.00% 0.00% 0.00% 0 IPC Seat Manager
 16         0          1         0 0.00% 0.00% 0.00% 0 IPC BackPressure
 17         0          1         0 0.00% 0.00% 0.00% 0 ARP Input
 18         4        314        12 0.00% 0.00% 0.00% 0 ARP Background
 19         0          2         0 0.00% 0.00% 0.00% 0 ATM Idle Timer
 20         4          2      2000 0.00% 0.00% 0.00% 0 AAA high-capacit
 21         0          1         0 0.00% 0.00% 0.00% 0 AAA_SERVER_DEADT
```

6.2 Ejecución

El script se lanza desde la terminal de Parrot-I con privilegios de root:

```
sudo python3 cdp-flooding.py -i ens4
```

Al iniciarse, el script confirma la interfaz seleccionada y comienza el envío continuo de frames CDP. Los dos indicadores de `pwntools` muestran en tiempo real el total de paquetes enviados y los datos del dispositivo falso inyectado en ese instante.

```
> python3 exploit.py
[*] Starting CDP DoS through the interface: ens4
[*] Press Ctrl+C to stop the attack.
[+] Send Packets: 3996
[q] Sending Packets: Device: RANDYNFC64XOV7 | MAC: 02:00:00:ac:45:54 | Port: Fas 1/1
```


6.3 Evidencia del impacto

Saturación de la tabla CDP

La tabla de vecinos CDP del router se inunda rápidamente con cientos de entradas falsificadas. Todas presentan el prefijo `RANDYN` en el Device ID, reportan entrada por `Fas 1/15` y simulan ser switches Cisco Catalyst WS-C2960. El router no puede distinguirlas de dispositivos legítimos y las procesa en su totalidad, llenando la tabla de vecinos.

```
R1#show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater
```

Device ID	Local Intrfce	Holdtme	Capability	Platform	Port ID
RANDYNXH7F3DOW	Fas 1/15	177	R S	WS-C2960-	Fas 1/2
RANDYN4B6XBGIZ	Fas 1/15	176	R S	WS-C2960-	Fas 3/0
RANDYN4AWQWEY6	Fas 1/15	176	R S	WS-C2960-	Fas 0/0
RANDYNLHV8Z3HK	Fas 1/15	173	R S	WS-C2960-	Fas 0/1
RANDYNFO00HNSR	Fas 1/15	173	R S	WS-C2960-	Fas 2/2
RANDYNPHCFXSRJ	Fas 1/15	171	R S	WS-C2960-	Fas 3/0
RANDYNNN7MM2BI	Fas 1/15	171	R S	WS-C2960-	Fas 3/3
RANDYNUKZ44ROW	Fas 1/15	170	R S	WS-C2960-	Fas 3/3
RANDYNO7SHC9UB	Fas 1/15	169	R S	WS-C2960-	Fas 1/0
RANDYNTMIS91LH	Fas 1/15	179	R S	WS-C2960-	Fas 1/1
RANDYN4WMIZ0GE	Fas 1/15	177	R S	WS-C2960-	Fas 2/0
RANDYNUTUU90DC	Fas 1/15	177	R S	WS-C2960-	Fas 3/0
RANDYNPMC98UTQ	Fas 1/15	173	R S	WS-C2960-	Fas 2/2
RANDYNV81SG79J	Fas 1/15	172	R S	WS-C2960-	Fas 3/0
RANDYNTJWAQGET	Fas 1/15	172	R S	WS-C2960-	Fas 0/0
RANDYNIRAI0L1Q	Fas 1/15	171	R S	WS-C2960-	Fas 3/1
RANDYN2IEHPC0Z	Fas 1/15	170	R S	WS-C2960-	Fas 3/3
RANDYN0MY1RJWU	Fas 1/15	169	R S	WS-C2960-	Fas 2/1
RANDYN5AAY46QV	Fas 1/15	169	R S	WS-C2960-	Fas 0/2
RANDYNB882IEA2	Fas 1/15	169	R S	WS-C2960-	Fas 3/0

Consumo de CPU

El procesamiento masivo de paquetes CDP lleva la utilización del CPU del router al 100% en modo de interrupciones, reflejado como `0%/100%` en el campo `five seconds` del comando `show processes cpu sorted`. El proceso `CDP Protocol` escala hasta liderar el ranking de consumo. Esta condición puede provocar lentitud severa en el procesamiento de otros protocolos, pérdida de conectividad de gestión o comportamiento inestable del equipo.


```

R1#show processes cpu sorted
CPU utilization for five seconds: 0%/100%; one minute: 0%; five minutes: 0%
  PID Runtime(ms)   Invoked    uSecs    5Sec    1Min    5Min  TTY Process
   85      272      3200        85    0.32%   0.10%   0.03%   0 CDP Protocol
   36       8       431        18    0.08%   0.00%   0.00%   0 Net Background
  149       0      3865         0    0.08%   0.00%   0.00%   0 RBSCP Background
    4       0         1         0    0.00%   0.00%   0.00%   0 EDDRI_MAIN
    2       0        79         0    0.00%   0.00%   0.00%   0 Load Meter
    3       0         1         0    0.00%   0.00%   0.00%   0 chkpt message ha
    7       0         2         0    0.00%   0.00%   0.00%   0 Timers
    1       0        67         0    0.00%   0.00%   0.00%   0 Chunk Manager
    5      28        39       717    0.00%   0.02%   0.00%   0 Check heaps
    6       0         1         0    0.00%   0.00%   0.00%   0 Pool Manager
    8       0         1         0    0.00%   0.00%   0.00%   0 OIR Handler
   12       4         1      4000    0.00%   0.00%   0.00%   0 IPC Zone Manager
   13       0       388         0    0.00%   0.00%   0.00%   0 IPC Periodic Tim
   14       0       388         0    0.00%   0.00%   0.00%   0 IPC Deferred Por
   15       0         1         0    0.00%   0.00%   0.00%   0 IPC Seat Manager
    9       0         1         0    0.00%   0.00%   0.00%   0 Crash writer
   17       0         1         0    0.00%   0.00%   0.00%   0 ARP Input
   10       0        14         0    0.00%   0.00%   0.00%   0 Environmental mo
   11       0         7         0    0.00%   0.00%   0.00%   0 IPC Dynamic Cach
   20       4         2      2000    0.00%   0.00%   0.00%   0 AAA high-capacit
   21       0         1         0    0.00%   0.00%   0.00%   0 AAA_SERVER_DEADT
--More--

```

7. Mitigación aplicada

Se documentan dos métodos de mitigación según el contexto de implementación.

7.1 Deshabilitar CDP de forma global

Desactiva el procesamiento CDP en todas las interfaces del dispositivo. Recomendado cuando CDP no es operacionalmente necesario en ningún segmento del equipo.

```

R1# configure terminal
R1(config)# no cdp run
R1(config)# end

```

```

R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#no cdp run
R1(config)#end
R1#

```

7.2 Deshabilitar CDP por interfaz

Desactiva CDP únicamente en la interfaz expuesta hacia zonas no confiables, preservando su funcionalidad en los enlaces de infraestructura donde sí sea requerida.

```
R1# configure terminal
R1(config)# interface FastEthernet 1/15
R1(config-if)# no cdp enable
R1(config-if)# end
```

```
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#int fa1/15
R1(config-if)#no cdp enable
R1(config-if)#end
```

8. Verificación post-mitigación

Tras aplicar cualquiera de las dos contramedidas, el router deja de procesar los frames CDP entrantes. Las entradas falsas en la tabla de vecinos se eliminan gradualmente a medida que expira su TTL (180 segundos), y el CPU regresa a valores normales.

Comandos de verificación recomendados:

```
R1# show cdp
R1# show cdp interface
R1# show cdp neighbors
R1# show processes cpu sorted
```

```
R1#show cdp
% CDP is not enabled
R1#show cdp interface
% CDP is not enabled
R1#show cdp neighbors
% CDP is not enabled
R1#
```

```
R1#show processes cpu sorted
CPU utilization for five seconds: 0%/0%; one minute: 0%; five minutes: 0%
PID Runtime(ms)   Invoked    uSecs   5Sec   1Min   5Min TTY Process
  1         0         68         0  0.00%  0.00%  0.00% 0 Chunk Manager
  2         0        275         0  0.00%  0.00%  0.00% 0 Load Meter
  3         0         1         0  0.00%  0.00%  0.00% 0 chkpt message ha
  4         0         1         0  0.00%  0.00%  0.00% 0 EDDRI_MAIN
  5         0        137         0  0.00%  0.00%  0.00% 0 Check heaps
  6         0         1         0  0.00%  0.00%  0.00% 0 Pool Manager
  7         0         2         0  0.00%  0.00%  0.00% 0 Timers
  8         0         1         0  0.00%  0.00%  0.00% 0 OIR Handler
  9         0         1         0  0.00%  0.00%  0.00% 0 Crash writer
 10         0         47         0  0.00%  0.00%  0.00% 0 Environmental mo
 11         0         23         0  0.00%  0.00%  0.00% 0 IPC Dynamic Cach
 12         0         1         0  0.00%  0.00%  0.00% 0 IPC Zone Manager
 13         4       1367         2  0.00%  0.00%  0.00% 0 IPC Periodic Tim
 14         0       1367         0  0.00%  0.00%  0.00% 0 IPC Deferred Por
 15         0         1         0  0.00%  0.00%  0.00% 0 IPC Seat Manager
 16         0         1         0  0.00%  0.00%  0.00% 0 IPC BackPressure
 17         0         1         0  0.00%  0.00%  0.00% 0 ARP Input
 18         0      1435         0  0.00%  0.00%  0.00% 0 ARP Background
 19         0         2         0  0.00%  0.00%  0.00% 0 ATM Idle Timer
 20         4         2      2000  0.00%  0.00%  0.00% 0 AAA high-capacit
 21         0         1         0  0.00%  0.00%  0.00% 0 AAA_SERVER_DEADT
```

9. Conclusión

El laboratorio confirma que CDP, al carecer de cualquier mecanismo de autenticación o validación de origen, puede ser abusado desde cualquier punto con acceso de Capa 2 para causar una denegación de servicio efectiva sobre un dispositivo Cisco. Con un script relativamente compacto basado en `scapy` y `pwntools`, fue posible elevar el CPU del router víctima al 100% en modo de interrupciones mediante el envío masivo de frames CDP falsificados con identidades completamente aleatorias.

La mitigación es directa y no invasiva: deshabilitar CDP donde no sea operacionalmente necesario. Este control debería aplicarse como configuración base en todo despliegue de dispositivos Cisco, especialmente en puertos de acceso orientados hacia usuarios finales o redes no administradas.

10. Referencias

- Repositorio del script: <https://github.com/RandyNin/DOS-CDP-FLOODING>
- Video demostrativo: https://youtu.be/9U_8M-r_Wf8
- Plataforma: GNS3 / Entorno controlado de Seguridad de Redes

Randy Nin / Matrícula 2025-0660